

Отчёт по лабораторной работе №13

Дисциплина: Операционные системы

Панина Жанна Валерьевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	8
4	Выполнение лабораторной работы	10
5	Ответы на контрольные вопросы	14
5.0.1	Каково предназначение команды <code>getopts</code> ?	14
5.0.2	Какое отношение метасимволы имеют к генерации имён файлов?	15
5.0.3	Какие операторы управления действиями вы знаете?	15
5.0.4	Какие операторы используются для прерывания цикла? . .	16
5.0.5	Для чего нужны команды <code>false</code> и <code>true</code> ?	16
5.0.6	Что означает строка <code>if test -f mans/i.s, <i>if test -f mans/i.s</i> проверяет, существует ли файл <code>mans/i.s</code> и является ли этот файл обычным файлом. Если данный файл является каталогом, то команда вернет нулевое значение (ложь).</code>	16
5.0.7	Объясните различия между конструкциями <code>while</code> и <code>until</code> . . .	16
6	Выводы	18
	Список литературы	19

Список иллюстраций

4.1	Создание файлов	10
4.2	Первый скрипт	11
4.3	Второй скрипт	12
4.4	Третий скрипт	13
4.5	Четвёртый скрипт	13

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами:
 - `-iinputfile` — прочитать данные из указанного файла;
 - `-ooutputfile` — вывести данные в указанный файл;
 - `-р`шаблон — указать шаблон для поиска;
 - `-C` — различать большие и малые буквы;
 - `-n` — выдавать номера строк, а затем ищет в указанном файле нужные строки, определяемые ключом `-р`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N (например `1.tmp`, `2.tmp`, `3.tmp`, `4.tmp` и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tag` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы

запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду find).

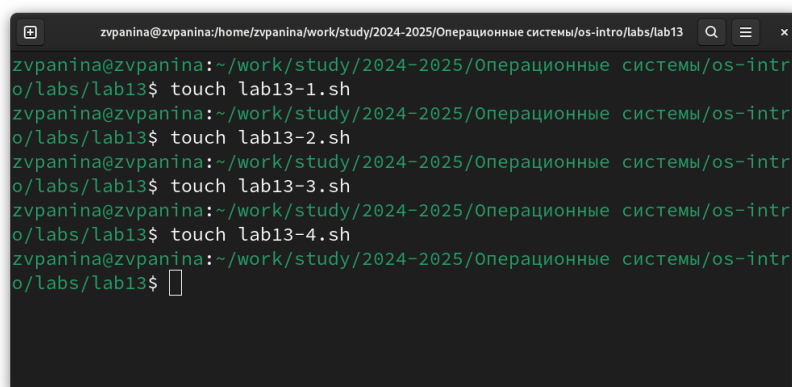
3 Теоретическое введение

Bash (Bourne Again Shell) — это мощная командная оболочка Unix, которая используется для выполнения различных задач в терминале. Bash предоставляет интерактивный интерфейс, в котором пользователи могут вводить команды, а затем получать результаты. Она также поддерживает скрипты оболочки, которые представляют собой текстовые файлы, содержащие последовательность команд Bash для автоматизации задач. Bash широко используется в средах Unix и Linux, а также поддерживается Windows с помощью подсистемы Windows для Linux (WSL). Перечислим основные возможности этой оболочки. Обработка команд. Bash может обрабатывать как простые, так и сложные команды. Простые состоят из одного действия и, возможно, некоторых аргументов. Сложные команды могут содержать несколько простых, объединенных с помощью операторов конвейера (`|`), перенаправления ввода и вывода (`<`, `>`, `>>`), условных операторов (`if/else`, `case/esac`, `while/do`). О них мы расскажем позже. Расширенный ввод, редактирование строк. Оболочка предоставляет функции расширенного ввода, такие как автодополнение, которое предлагает возможные варианты завершения команд и имен файлов по мере их ввода. Она также поддерживает историю, позволяя пользователям просматривать ранее введенные команды, а затем повторно их использовать. Возможность создания и запуска скриптов. Скрипты оболочки — это текстовые файлы, содержащие последовательность команд. Их можно создавать с помощью текстового редактора, а затем запускать в терминале, что дает возможность пользователям автоматизировать задачи и управлять системой. Скрипты могут содержать условные операторы, циклы, функции для обеспече-

ния дополнительной гибкости и контроля.

4 Выполнение лабораторной работы

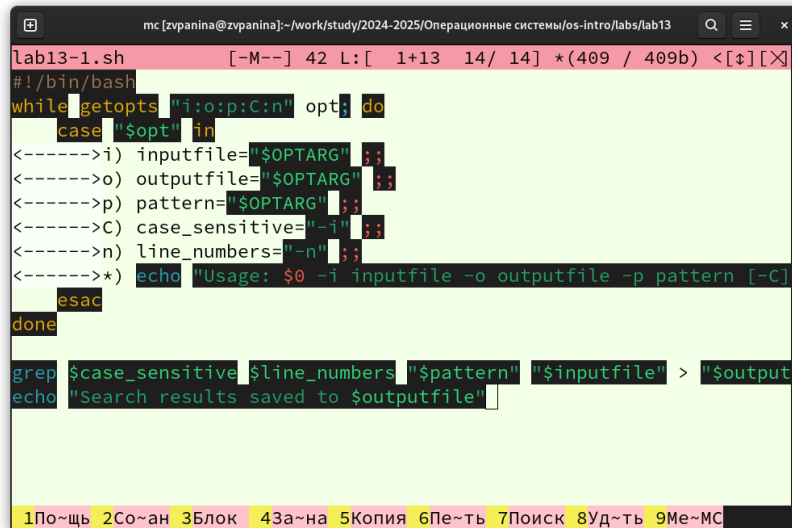
1. Создаю четыре файла .sh для скриптов и открываю тс для редактирования(рис. 4.1).



```
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13$ touch lab13-1.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13$ touch lab13-2.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13$ touch lab13-3.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13$ touch lab13-4.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13$
```

Рис. 4.1: Создание файлов

2. Используя команды `getopts` `grep`, пишу командный файл, который анализирует командную строку с ключами:
 - `-iinputfile` — прочитать данные из указанного файла;
 - `-ooutputfile` — вывести данные в указанный файл;
 - `-ршаблон` — указать шаблон для поиска;
 - `-С` — различать большие и малые буквы;
 - `-п` — выдавать номера строк, а затем ищет в указанном файле нужные строки, определяемые ключом `-р`. (рис. 4.2).



```
lab13-1.sh [-M--] 42 L: [ 1+13 14/ 14] *(409 / 409b) <[+][X]
#!/bin/bash
while getopts "i:o:p:C:n" opt; do
  case "$opt" in
    <----->i) inputfile="$OPTARG" ;;
    <----->o) outputfile="$OPTARG" ;;
    <----->p) pattern="$OPTARG" ;;
    <----->C) case_sensitive="-i" ;;
    <----->n) line_numbers="-n" ;;
    <----->*) echo "Usage: $0 -i inputfile -o outputfile -p pattern [-C]"
  esac
done

grep $case_sensitive $line_numbers "$pattern" "$inputfile" > "$output"
echo "Search results saved to $outputfile"
```

Рис. 4.2: Первый скрипт

- Пишу на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено. (рис. 4.3).

```
mc [zypanina@zypanina]:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab13
lab13-2.sh [-M--] 2 L: [ 1+24 25/ 25] *(458 / 458b) <E[+] [X]
#!/bin/bash

cat <& EOF > check_number.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num > 0) exit(1);
    if (num < 0) exit(2);
    exit(0);
}
EOF

gcc check_number.c -o check_number
./check_number
status=$?
if [ $status -eq 1 ]; then
    echo "The number is positive."
elif [ $status -eq 2 ]; then
    echo "The number is negative."
else
    echo "The number is zero."
fi
```

Рис. 4.3: Второй скрипт

4. Пишу командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют). (рис. 4.4).

Рис. 4.4: Третий скрипт

5. Пишу командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду find). (рис. 4.5)

Рис. 4.5: Четвёртый скрипт

5 Ответы на контрольные вопросы

5.0.1 Каково предназначение команды `getopts`?

Осуществляет синтаксический анализ командной строки, выделяя флаги, и используется для объявления переменных. Синтаксис команды следующий: `getopts option-string variable`. Флаги – это опции командной строки, обычно помеченные знаком минус; Например, `-F` является флагом для команды `ls -F`. Иногда эти флаги имеют аргументы, связанные с ними. Программы интерпретируют эти флаги, соответствующим образом изменяя свое поведение. Строка опций `option-string` – это список возможных букв и чисел соответствующего флага. Если ожидается, что некоторый флаг будет сопровождаться некоторым аргументом, то за этой буквой должно следовать двоеточие. Соответствующей переменной присваивается буква данной опции. Если команда `getopts` может распознать аргумент, она возвращает истину. Принято включать `getopts` в цикл `while` и анализировать введенные данные с помощью оператора `case`. Предположим, необходимо распознать командную строку следующего формата: `testprog -infile_in.txt -outfile_out.doc -L -t -r` Вот как выглядит использование оператора `getopts` в этом случае:

```
while
getopts o:i:Ltr optletter do
case optletter in
o) iflag = 1; oval =OPTARG;;
i) iflag=1;
ival=$OPTARG;;
L) Lflag=1;;
t) tflag=1;;
r) rflag=1;;
*) echo Illegal option $optletter
esac
done
```

 Функция `getopts` включает две специальные переменные среды – `OPTARG` и `OPTIND`. Если ожидается дополнительное значение, то `OPTARG` устанавливается в значение этого аргумента (будет равна `file_in.txt` для опции `i` и `file_out.doc` для опции `o`). `OPTIND` является числовым индексом на упомянутый аргумент. Функция `getopts` также понимает переменные типа массив, следовательно, можно

использовать ее в функции не только для синтаксического анализа аргументов функций, но и для анализа введенных пользователем данных.

5.0.2 Какое отношение метасимволы имеют к генерации имён файлов?

При перечислении имён файлов текущего каталога можно использовать следующие символы: – соответствует произвольной, в том числе и пустой строке; ? – соответствует любому одинарному символу; [c1-c2] – соответствует любому символу, лексикографически находящемуся между символами c1 и c2. Например, `echo *` – выведет имена всех файлов текущего каталога, что представляет собой простейший аналог команды `ls`; `ls .c` – выведет все файлы с последними двумя символами, совпадающими с `.c`. `echo prog.?` – выведет все файлы, состоящие из пяти или шести символов, первыми пятью символами которых являются `prog.` `[a-z]` – соответствует произвольному имени файла в текущем каталоге, начинающемуся с любой строчной буквы латинского алфавита.

5.0.3 Какие операторы управления действиями вы знаете?

Часто бывает необходимо обеспечить проведение каких-либо действий циклически и управление дальнейшими действиями в зависимости отрезультатов проверки некоторого условия. Для решения подобных задач язык программирования `bash` предоставляет возможность использовать такие управляющие конструкции, как `for`, `case`, `if` и `while`. С точки зрения командного процессора эти управляющие конструкции являются обычными командами и могут использоваться как при создании командных файлов, так и при работе в интерактивном режиме. Команды, реализующие подобные конструкции, по сути, являются операторами языка программирования `bash`. Поэтому при описании языка программирования `bash` термин оператор будет использоваться наравне с термином команда. Команды ОС UNIX возвращают код завершения, значение которого мо-

жет быть использовано для принятия решения о дальнейших действиях. Команда `test`, например, создана специально для использования в командных файлах. Единственная функция этой команды заключается в выработке кода завершения.

5.0.4 Какие операторы используются для прерывания цикла?

Два несложных способа позволяют вам прерывать циклы в оболочке `bash`. Команда `break` завершает выполнение цикла, а команда `continue` завершает данную итерацию блока операторов. Команда `break` полезна для завершения цикла `while` в ситуациях, когда условие перестаёт быть правильным. Команда `continue` используется в ситуациях, когда больше нет необходимости выполнять блок операторов, но вы можете захотеть продолжить проверять данный блок на других условных выражениях.

5.0.5 Для чего нужны команды `false` и `true`?

Следующие две команды ОС UNIX используются только совместно с управляющими конструкциями языка программирования `bash`: это команда `true`, которая всегда возвращает код завершения, равный нулю (т.е. истина), и команда `false`, которая всегда возвращает код завершения, не равный нулю (т. е. ложь).

5.0.6 Что означает строка `if test -f mans/i.s, ?if test -f mans/i.s`

проверяет, существует ли файл `mans/i.s` и является ли этот файл обычным файлом. Если данный файл является каталогом, то команда вернет нулевое значение (ложь).

5.0.7 Объясните различия между конструкциями `while` и `until`.

Выполнение оператора цикла `while` сводится к тому, что сначала выполняется последовательность команд (операторов), которую задаёт список-команд

в строке, содержащей служебное слово `while`, а затем, если последняя выполненная команда из этой последовательности команд возвращает нулевой код завершения (истина), выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `do`, после чего осуществляется безусловный переход на начало оператора цикла `while`. Выход из цикла будет осуществлён тогда, когда последняя выполненная команда из последовательности команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, возвратит ненулевой код завершения (ложь). При замене в операторе цикла `while` служебного слова `while` на `until` условие, при выполнении которого осуществляется выход из цикла, меняется на противоположное. В остальном оператор цикла `while` и оператор цикла `until` идентичны.

6 Выводы

В ходе выполнения лабораторной работы я изучила основы программирования в оболочке ОС UNIX. Научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Список литературы